PRD: Neoke Wallet Web Client — Layer 3: Consent Engine Integration

Full Implementation Specification for an AI Developer with No Prior Context

Document version: 1.0
Written for: Claude Code (AI developer — assume zero knowledge of SSI, digital identity, the Neoke platform, or the existing wallet codebase)
Codebase dependency: Existing wallet web client (React 18 / TypeScript / Vite) — you will extend it, not replace it
Service dependency: Neoke IDN Node HTTP API (Layer 1) and Neoke Consent Engine HTTP API (Layer 2)
Delivery: Extended version of the existing wallet, no new repo

---

SECTION 0 — READ THIS FIRST: FULL CONTEXT FOR A DEVELOPER WITH ZERO PRIOR KNOWLEDGE

This is a long section. Do not skip it. You need to understand the full domain before touching a single file.

0.1 What Is a Digital Credential Wallet?

A digital credential wallet is a web application that allows a person or organisation to:

1. **Receive** signed digital documents from issuers (universities, governments, employers) — called **credentials**. Examples: a driving licence, an employee badge, an age verification proof.
2. **Store** those credentials securely.
3. **Present** selected credentials (or selected fields from them) to verifiers (border controls, shops, partner services) — called a **presentation**.

Credentials are cryptographically signed. The holder cannot forge them, and any verifier can check the issuer's signature without contacting the issuer. Think of it as a tamper-proof digital passport.

The wallet in this project is a **web application** — a React SPA running in a browser. It connects to a **Neoke IDN Node** (a cloud-hosted identity server) via an HTTP API. The node holds the actual cryptographic keys and signed credentials. The wallet is the UI layer that lets a human trigger and observe operations on that node.

0.2 What Is the Neoke IDN Node (Layer 1)?

The IDN Node is an external HTTP API service. It is a black box — you never see its source code or touch its internals. You only call its REST API.

Key API endpoints used by the wallet:

Endpoint	Method	Purpose
`/:/auth/authn`	POST	Exchange an API key for a short-lived bearer token
`/:/credentials/stored`	GET	Fetch all credentials held in the node's wallet
`/:/credentials/stored/{id}`	DELETE	Delete a specific credential
`/:/credentials/types`	GET	Fetch the node's credential type registry
`/:/oid4vci/receive`	POST	Accept a credential offer (receive a new credential)
`/:/auth/siop/respond/preview`	POST	Preview a VP request — see what would be shared, without sharing
`/:/auth/siop/respond`	POST	Actually respond to a VP request (share credentials)
`/:/keys`	GET	List cryptographic keys in the node wallet

Authentication: Every call (except the auth exchange itself) requires:

text
```
Authorization: Bearer {token}
```

Tokens are obtained from `POST /:/auth/authn` using:

text
```
Authorization: ApiKey {your-api-key}
```

Tokens expire. The wallet watches expiry and triggers a re-authentication modal when the token is about to expire.

Node identifier: Each node has a short identifier like `b2b-poc`. This resolves to a full URL: `https://b2b-poc.id-node.neoke.com`. The wallet's existing `nodeIdentifierToUrl()` function handles this mapping.

0.3 What Are the Credential Formats?

Two formats are in use:

mso_mdoc (Mobile Document / ISO 18013-5): Used for government-style identity documents (driving licences, Photo IDs). Claims are organised into *namespaces* — a two-level map like:

json
```
{ "org.iso.18013.5.1": { "family_name": "Smith", "portrait": "..." } }
```

dc+sd-jwt (Selective Disclosure JWT): Used for privacy-sensitive credentials. Each field can be independently hidden or revealed. Claims are a flat JSON object:

json
```
{ "over_18": true, "nationality": "NL" }
```

Both formats are received via the same issuance flow and presented via the same verification flow. The wallet handles both transparently.

0.4 What Are Credential Offer URIs and VP Request URIs?

Credential offer URI (issuance): A special deep link an issuer sends to invite a wallet to receive a credential.

text
```
openid-credential-offer://?credential_offer_uri=https%3A%2F%2F...
```

VP request URI (verification): A special deep link a verifier sends to request credential presentation.

text
```
openid4vp://?client_id=did%3Aweb%3A...&request_uri=https%3A%2F%2F...
```

Both arrive as URL query parameters when the user follows a link:

text
```
https://wallet.example.com/?uri=openid4vp%3A%2F%2F%3F...
```

The wallet's existing `detectUriType()` classifies any raw string as `receive`, `present`, or `unknown`. The classification determines which screen handles the link.

0.5 How Does the Existing Wallet Work Today?

The existing wallet is built with React 18, TypeScript, Tailwind CSS, and Vite. It has no routing library — navigation is a simple `currentView` state variable in `App.tsx` with a `navigate()` function. There is no URL-based routing.

Authentication flow:

1. Onboarding Step 1: User enters a node identifier (e.g., `b2b-poc`). The wallet validates the node is reachable.
2. Onboarding Step 2: User enters their API key. The wallet calls `POST /:/auth/authn`, receives a bearer token, and stores it in `localStorage` with an expiry timestamp.
3. `AuthContext` manages token state, expiry monitoring, and session restore on page reload.

Main screens:

- `DashboardScreen` — Shows the credential card carousel and navigates to credential detail.
- `CredentialDetailScreen` — Shows full fields of a selected credential.
- `ReceiveScreen` — Handles credential offer URIs end-to-end (shows offer details, calls `/:/oid4vci/receive`).

- `PresentScreen` — Handles VP request URIs end-to-end (calls preview, shows what will be shared, calls respond on user approval).
- `AccountScreen` — Shows current node connection, logout.
- `OnboardingStep1Screen` / `OnboardingStep2Screen` — Two-step login flow.
- `ReAuthModal` — Overlay shown when token expires mid-session.

Bottom tab bar: Three tabs — Home (dashboard), Scan QR Code (navigates to receive), Account.

Deep link routing: On mount, `App.tsx` reads `?uri=` or `?offer_uri=` from `window.location.search`, classifies the URI, and after authentication, navigates directly to `ReceiveScreen` or `PresentScreen` with the URI as `initialUri`.

State storage: `localStorage` keys:

- `neoke_s_token` — bearer token
- `neoke_s_expires` — token expiry (ms timestamp)
- `neoke_node_id` — node identifier (short form)
- `neoke_activity` — last activity timestamp (for 7-day inactivity lock)
- `neoke_credentials` — locally cached credential list (merged server/local)

Existing API client: `src/api/client.ts` exposes all node API calls. The module-level `_baseUrl` is set by `setBaseUrl()` during onboarding and session restore. Every API function takes a `token` string.

0.6 What Is the Consent Engine (Layer 2) and Why Does the Wallet Need to Know About It?

The **Consent Engine** is a separate HTTP microservice (a Node.js/TypeScript process running at its own URL, e.g., `https://consent.example.com`). It was built as Layer 2 and sits between the wallet and the IDN Node for automated, policy-driven decisions.

What it does (relevant to the wallet):

1. **Intake:** It accepts any credential-related URI (VP request, credential offer, verification link) and evaluates it against a database of user-defined rules.
2. **Automated decisions:** If a matching rule exists, it acts automatically:
    - For VP requests: calls the IDN Node's preview → respond on the user's behalf.
    - For credential offers: calls the IDN Node's receive on the user's behalf.
    - Returns `action: "auto_executed"` immediately.

3. **Queue:** If no rule matches, it puts the request into a pending queue and returns `action: "queued"`. The user must later approve or reject it.
4. **Rejection:** If a request cannot be handled (no credentials match, offer expired, etc.), it returns `action: "rejected"`.
5. **Rules management:** It exposes a full CRUD API for managing the rules that drive its decisions.
6. **Audit log:** It records every decision (auto or manual) with full context.

Why the wallet needs to integrate:

- Without Layer 3, the wallet is unaware of the Consent Engine. Users still process every link manually (scan → see → approve → done). This is fine for interactive use but misses the automation benefit.
- With Layer 3, the wallet becomes the **management UI for the Consent Engine**: users create rules, review queued requests, see audit history, and observe automated decisions — all from the same wallet they already use.
- Additionally, deep links that arrive in the wallet should be **routed through the CE** (when CE is configured) instead of processed directly, so that rules are enforced even for links the user opens manually.

Key principle: The Consent Engine is **optional**. If the user has not configured a CE URL, the wallet behaves exactly as it does today — direct mode, no changes to existing flows. Every new CE-specific feature is additive and non-breaking.

0.7 How Does the Consent Engine API Work?

The CE exposes a REST API at its configured base URL. Authentication uses the same API key mechanism as the IDN Node:

text
```
Authorization: ApiKey {api-key}
```

The CE validates this key against its configured IDN Node. The same API key the user entered at wallet onboarding (for the IDN Node) is also valid for the CE, because the CE authenticates by verifying the key against the node it is bound to. **No separate CE password is needed.**

Core intake endpoint:

text
```
POST {ceBaseUrl}/consent/intake
Authorization: ApiKey {api-key}
```

```
Content-Type: application/json

{
  "rawLink": "openid4vp://?client_id=...&request_uri=..."
}
```

Response:

json
```
{
  "requestId": "uuid",
  "action": "auto_executed",
  "linkType": "vp_request",
  "reason": null
}
```

Possible `action` values:

- `"auto_executed"` — a matching rule was found and the operation was performed automatically.
- `"queued"` — no rule matched; the request is now in the pending queue.
- `"rejected"` — the request was rejected (no matching credentials, expired link, etc.).
- `"manual_required"` — the CE was unable to process this link type; the wallet should handle it directly using the existing flow. This is the fallback for edge cases the CE cannot handle.

Rules API (abbreviated):

- `GET /rules` — list all rules
- `POST /rules` — create a rule
- `GET /rules/{id}` — get a rule
- `PATCH /rules/{id}` — update a rule
- `DELETE /rules/{id}` — delete a rule
- `POST /rules/{id}/enable` — enable a rule
- `POST /rules/{id}/disable` — disable a rule
- `POST /rules/{id}/test` — dry-run: returns `wouldMatch: true/false` for a given link without executing

Queue API:

- `GET /queue` — list pending requests (filter by `status: "pending"`)
- `GET /queue/{id}` — get one pending request with full context (verifier info, requested fields, etc.)
- `POST /queue/{id}/approve` — approve and execute
- `POST /queue/{id}/reject` — reject without executing
- `DELETE /queue/{id}` — delete from queue

Audit API:

- `GET /audit` — list audit events with optional filtering (`?nodeId=`, `?action=`, `?limit=`, `?offset=`)

Discovery API (for rule creation wizard):

- `GET /node/{nodeId}/credentials` — list credential types and their available claims from the node

Health:

- `GET /health` — returns `{ status: "ok" | "degraded", nodeConnected: boolean, pendingCount: number }`

0.8 What Is a Consent Rule? (Explained Simply)

A consent rule is a statement like:

> "Whenever service X asks for fields A and B from my Employee Credential, automatically share them — but only on weekdays."

Technically, a rule has:

typescript
```typescript
interface ConsentRule {
  id: string;                // UUID, assigned by CE
  nodeId: string;            // Which node this rule applies to
  ruleType: "verification" | "issuance";  // VP request or credential
offer
  enabled: boolean;

  // Who is allowed (for verification rules)
  party: {
```

```
    matchType: "any" | "did" | "domain" | "domain_wildcard";
    value?: string;         // e.g. "did:web:example.com" or
"*.airline.com"
  };

  // What credential type (or "any")
  credentialType: string | "any";
  credentialFormat?: string | "any";

  // Which fields are allowed
  allowedFields: {
    mode: "any" | "explicit";
    explicit?: string[];    // e.g. ["family_name", "given_name"]
  };

  // Optional extra conditions
  conditions?: Array<{
    type: "time_of_day" | "day_of_week" | "max_per_day" |
"require_linked_domain";
    // ...parameters depend on type
  }>;

  // Expiry
  expiry: {
    type: "never" | "date" | "uses";
    expiresAt?: string;     // ISO date for type: "date"
    maxUses?: number;
    usedCount?: number;
  };

  // For issuance rules: which issuer to trust
  trustedIssuerDid?: string | "any";

  // Metadata
  label?: string;           // Human-readable name, e.g. "My airline
check-in rule"
  createdAt: string;
  updatedAt: string;
```

```
}
```

A rule with `party.matchType: "any"`, `credentialType: "any"`, and `allowedFields.mode: "any"` is a **catch-all** — it allows the CE to auto-process any request. More specific rules override less specific ones when both match.

0.9 What Is a Pending Queue Request?

When a VP request or credential offer arrives and no rule matches, the CE stores it as a **pending request**:

typescript
```
interface PendingRequest {
  id: string;
  nodeId: string;
  linkType: "vp_request" | "credential_offer";
  status: "pending" | "approved" | "rejected" | "expired";
  createdAt: string;
  expiresAt: string;       // TTL from when it was queued (e.g., 24
hours)

  // Context for the user to make a decision
  preview: {
    // For VP requests:
    verifier?: {
      clientId: string;    // DID of the verifier
      name?: string;
      linkedDomains?: string[];
    };
    requestedFields?: string[];      // Field names
    credentialType?: string;
    matchedCredentials?: Array<{ id: string; type: string; issuer:
string }>;

    // For credential offers:
    issuer?: string;       // DID of the issuer
    credentialTypes?: string[];
```

```
    requiresPin?: boolean;
  };
}
```

The user opens the queue, reads the context, and either approves (CE executes) or rejects (CE discards).

0.10 What Is an Audit Event?

Every action the CE takes is recorded:

typescript
```
interface AuditEvent {
  id: string;
  nodeId: string;
  requestId: string;
  action: "auto_presented" | "auto_received" | "manually_approved" |
"manually_rejected" | "queued" | "rejected" | "expired";
  linkType: "vp_request" | "credential_offer";
  ruleId?: string;         // Which rule fired (if any)
  ruleLabel?: string;
  verifierDid?: string;
  credentialType?: string;
  requestedFields?: string[];
  timestamp: string;
}
```

The audit log is read-only from the wallet's perspective.

0.11 The Real-World Problem Layer 3 Solves

Without Layer 3, even though the CE (Layer 2) exists and has rules configured, the wallet user has no way to:

- Create or manage those rules through a UI.

- See what is in the pending queue.
- Approve or reject queued requests from their wallet.
- Know that a request was auto-processed on their behalf.
- Route links they open manually through the CE.

Layer 3 bridges the CE's backend capabilities with a complete, friendly UI embedded in the wallet the user already knows.

SECTION 1 — WHAT YOU ARE BUILDING

Layer 3 is an **extension of the existing wallet web client**. You do not create a new repository. You add new files and modify existing ones within the same React/TypeScript/Vite project.

Layer 3 adds:

1. **Consent Engine configuration** — a new section in the Account screen and onboarding where the user enters a CE URL.
2. **CE API client** — a new `src/api/consentEngineClient.ts` module mirroring the structure of `src/api/client.ts`.
3. **CE context** — a new `ConsentEngineContext` managing CE connection state and pending queue count badge.
4. **Modified deep-link routing** — when a URI arrives and CE is configured, route it through `POST /consent/intake` instead of directly to `ReceiveScreen` or `PresentScreen`. Display the CE's decision to the user.
5. **Five new screens** — `ConsentRulesScreen`, `ConsentRuleEditorScreen`, `ConsentQueueScreen`, `ConsentQueueDetailScreen`, `AuditLogScreen`.
6. **Updated navigation** — a new "Consent" tab in the bottom tab bar (shown only when CE is configured), and a badge on the tab when there are pending queue items.
7. **Updated `types/index.ts`** — new `ViewName` values for CE screens.

SECTION 2 — EXISTING CODEBASE REFERENCE

Before writing any new code, understand the existing files precisely.

2.1 `src/App.tsx`

The root component. Contains:

- `TabBar` component (three buttons: Home, Scan QR Code, Account).
- `AppInner` component — handles onboarding, deep-link extraction from `window.location.search`, navigation state, and renders the current screen.
- `App` root — wraps `AppInner` in `AuthProvider`.

Key state in `AppInner`:

- `currentView: ViewName` — which screen is shown.
- `deepLinkUri: string | null` — extracted once on mount from URL query string.
- `pendingUri: string | undefined` — passed as `initialUri` prop to `ReceiveScreen` / `PresentScreen`.
- `navigate(view, extra?)` — sets `currentView`, `selectedCredential`, `pendingUri`.

Deep-link detection logic handles both URL-encoded and raw (un-encoded) URI forms, and two param names (`uri` and `offer_uri`). This logic must not be broken.

2.2 `src/api/client.ts`

Module-level mutable `_baseUrl` set by `setBaseUrl()`. Every API function takes a `token` string. The `request<T>()` internal function handles auth header injection, error parsing, and response parsing. `ApiError` is the custom error class with `status` and `body` fields.

Key functions used by Layer 3:

- `apiKeyAuth(apiKey, nodeBaseUrl?)` — exchange API key for bearer token.
- `nodeIdentifierToUrl(identifier)` — map short ID to full URL.

2.3 `src/context/AuthContext.tsx`

Provides `state.token`, `state.nodeIdentifier`, `state.baseUrl`, `state.sessionExpired`, `state.loginNonce`. Also provides `setNode`, `setToken`, `logout`, `markExpired`.

The token is a bearer token for the **IDN Node**. It is also the credential that the CE validates (the CE calls `/:/auth/authn` on the node to verify the API key). So the API key entered at onboarding is the same credential used to authenticate with both the node and the CE.

2.4 `src/utils/uriRouter.ts`

`detectUriType(uri): "receive" | "present" | "unknown"`. This is used for deep-link classification. Layer 3 must preserve this and use it as the basis for CE intake routing.

2.5 `src/store/localCredentials.ts`

`localStorage`-backed credential cache with `getLocalCredentials`, `saveLocalCredential`, `deleteLocalCredential`, `clearLocalCredentials`, `mergeWithLocalCredentials`.

2.6 `src/types/index.ts`

All TypeScript interfaces. Currently `ViewName` is:

typescript
```
export type ViewName = 'login' | 'dashboard' | 'detail' | 'receive' |
'present' | 'account';
```

You will extend this.

SECTION 3 — CE CONFIGURATION AND STORAGE

3.1 New `localStorage` Keys

Add these to a dedicated constants file or alongside the existing `SK` constants in `AuthContext`:

typescript
```
const CE_SK = {
  CE_URL:     'neoke_ce_url',      // Consent Engine base URL (e.g.,
"https://consent.example.com")
  CE_ENABLED: 'neoke_ce_enabled',  // "true" or "false" — user-level
toggle
} as const;
```

The CE URL is stored in `localStorage` separately from `AuthContext`. It is scoped to the node — if the user switches nodes (logs out and into a different node), the CE URL from the

previous session should be cleared. To enforce this, clear `neoke_ce_url` and `neoke_ce_enabled` in `AuthContext`'s `logout()` handler.

3.2 CE Configuration Rules

- CE is **always optional**. If `neoke_ce_url` is absent or empty, the wallet behaves exactly as today.
- The CE URL must be a valid `https://` URL. Validate with a simple check: starts with `https://` and contains at least one `.` after the scheme.
- The CE API key is the **same API key** the user entered at onboarding Step 2. It is not stored separately in `localStorage` (it is kept only in memory as part of the auth flow). To authenticate with the CE, the wallet uses `Authorization: ApiKey {apiKey}` — but this requires the raw API key. **Problem:** the wallet does not retain the raw API key after onboarding (it only stores the resulting bearer token).

Solution: When the user configures the CE URL (in AccountScreen), ask them to enter their API key once more. Store it encrypted in `localStorage` under `neoke_ce_apikey` using the same simple approach used for session tokens (no deep encryption is needed for MVP — localStorage is the boundary). This is consistent with the Layer-2 PRD's note that API keys can be treated as plaintext in controlled environments.

Alternatively (simpler for MVP): add a step to onboarding Step 2 where, if a CE URL has already been configured from a previous session, the CE API key is refreshed automatically by re-using the key entered in the current session.

Recommended implementation: Store the raw API key in `localStorage` under `neoke_ce_apikey` at the time the user completes onboarding Step 2. Clear it on logout. This makes the key available whenever the CE needs to be called, without re-prompting the user.

SECTION 4 — CE API CLIENT (`src/api/consentEngineClient.ts`)

Create a new file that mirrors the structure of `src/api/client.ts` but targets the CE.

4.1 Module-Level State

typescript
```
let _ceBaseUrl = '';
```

```
export function setCeBaseUrl(url: string): void {
  _ceBaseUrl = url.replace(/\/$/, '');
}

export function getCeBaseUrl(): string {
  return _ceBaseUrl;
}

export function isCeConfigured(): boolean {
  return _ceBaseUrl.length > 0;
}
```

4.2 Error Handling

Reuse `ApiError` from `src/api/client.ts` (import it) or create a separate `CeApiError` class — either is fine. The error format from the CE is:

```
{
  "error": {
    "code": "RULE_NOT_FOUND",
    "message": "...",
    "requestId": "...",
    "timestamp": "..."
  }
}
```

Map this into `CeApiError` with `code` and `message` fields.

4.3 Internal `ceRequest<T>()` Function

Signature:

```typescript
async function ceRequest<T>(
  path: string,
  options: RequestInit & { apiKey?: string } = {}
```

```
): Promise<T>
```

- Uses `_ceBaseUrl` as base.
- Injects `Authorization: ApiKey {apiKey}` header when `apiKey` is provided.
- Throws `CeApiError` on non-2xx responses with structured error parsing.
- Throws a connectivity error if `_ceBaseUrl` is not set.

4.4 Exported Functions

typescript

```typescript
// Health check — no auth required
export async function checkCeHealth(): Promise<{
  status: 'ok' | 'degraded';
  nodeConnected: boolean;
  pendingCount: number;
}>;

// Validate CE URL: any HTTP response (even 4xx) means it's reachable
export async function validateCeUrl(url: string): Promise<void>;

// Intake: route a raw link through the CE
export async function ceIntake(
  apiKey: string,
  rawLink: string
): Promise<{
  requestId: string;
  action: 'auto_executed' | 'queued' | 'rejected' | 'manual_required';
  linkType: 'vp_request' | 'credential_offer' | 'verification_link' |
'unknown';
  reason?: string;
}>;

// Rules
export async function listRules(apiKey: string):
Promise<ConsentRule[]>;
export async function createRule(apiKey: string, rule:
CreateRulePayload): Promise<ConsentRule>;
```

```
export async function updateRule(apiKey: string, id: string, patch:
Partial<CreateRulePayload>): Promise<ConsentRule>;
export async function deleteRule(apiKey: string, id: string):
Promise<void>;
export async function enableRule(apiKey: string, id: string):
Promise<ConsentRule>;
export async function disableRule(apiKey: string, id: string):
Promise<ConsentRule>;
export async function testRule(
  apiKey: string,
  id: string,
  rawLink: string
): Promise<{ wouldMatch: boolean; rule: ConsentRule }>;

// Queue
export async function listQueue(
  apiKey: string,
  options?: { status?: string; limit?: number; offset?: number }
): Promise<{ items: PendingRequest[]; total: number }>;
export async function getQueueItem(apiKey: string, id: string):
Promise<PendingRequest>;
export async function approveQueueItem(
  apiKey: string,
  id: string,
  txCode?: string
): Promise<{ status: string }>;
export async function rejectQueueItem(
  apiKey: string,
  id: string,
  reason?: string
): Promise<{ status: string }>;
export async function deleteQueueItem(apiKey: string, id: string):
Promise<void>;

// Audit
export async function listAuditEvents(
  apiKey: string,
  options?: { limit?: number; offset?: number; action?: string }
```

```
): Promise<{ events: AuditEvent[]; total: number }>;

// Discovery (credential types + claims from the node, via CE)
export async function listNodeCredentialTypes(
  apiKey: string,
  nodeId: string
): Promise<NodeCredentialType[]>;
```

SECTION 5 — NEW TYPES (`src/types/consentEngine.ts`)

Create a new file with all CE-related types. Do not mix them into `src/types/index.ts` — keep them separate for clarity.

typescript

```
// ============================================================
// Consent Rules
// ============================================================

export type RuleType = 'verification' | 'issuance';
export type PartyMatchType = 'any' | 'did' | 'domain' |
'domain_wildcard';
export type FieldsMode = 'any' | 'explicit';
export type ExpiryType = 'never' | 'date' | 'uses';
export type ConditionType = 'time_of_day' | 'day_of_week' |
'max_per_day' | 'require_linked_domain';

export interface RuleParty {
  matchType: PartyMatchType;
  value?: string;
}

export interface RuleAllowedFields {
  mode: FieldsMode;
  explicit?: string[];
}
```

```
export interface RuleCondition {
  type: ConditionType;
  startHour?: number;       // for time_of_day: 0–23
  endHour?: number;         // for time_of_day: 0–23
  allowedDays?: number[];   // for day_of_week: 0=Sun, 1=Mon, ...,
6=Sat
  limit?: number;           // for max_per_day
}

export interface RuleExpiry {
  type: ExpiryType;
  expiresAt?: string;       // ISO date string, for type: "date"
  maxUses?: number;         // for type: "uses"
  usedCount?: number;       // maintained by CE, read-only
}

export interface ConsentRule {
  id: string;
  nodeId: string;
  ruleType: RuleType;
  enabled: boolean;
  label?: string;
  party: RuleParty;
  credentialType: string;   // "any" or a specific type ID
  credentialFormat?: string; // "any", "mso_mdoc", "dc+sd-jwt",
"jwt_vc_json"
  allowedFields: RuleAllowedFields;
  conditions?: RuleCondition[];
  expiry: RuleExpiry;
  trustedIssuerDid?: string; // for issuance rules: "any" or specific
DID
  txCode?: string;           // for issuance rules: optional PIN
  createdAt: string;
  updatedAt: string;
}
```

```
export type CreateRulePayload = Omit<ConsentRule, 'id' | 'createdAt' |
'updatedAt' | 'expiry'> & {
  expiry: Omit<RuleExpiry, 'usedCount'>;
};

// ==============================================================
// Pending Queue
// ==============================================================

export type RequestStatus = 'pending' | 'approved' | 'rejected' |
'expired';
export type LinkType = 'vp_request' | 'credential_offer' |
'verification_link';

export interface QueueItemPreview {
  // For VP requests
  verifier?: {
    clientId: string;
    name?: string;
    purpose?: string;
    linkedDomains?: string[];
  };
  requestedFields?: string[];
  credentialType?: string;
  credentialFormat?: string;
  matchedCredentials?: Array<{ id: string; type: string; issuer:
string }>;

  // For credential offers
  issuerDid?: string;
  credentialTypes?: string[];
  requiresPin?: boolean;
}

export interface PendingRequest {
  id: string;
  nodeId: string;
  linkType: LinkType;
```

```
  status: RequestStatus;
  rawLink: string;
  preview: QueueItemPreview;
  createdAt: string;
  expiresAt: string;
  resolvedAt?: string;
  resolvedAction?: 'approved' | 'rejected';
  rejectReason?: string;
}

// ================================================================
// Audit
// ================================================================

export type AuditAction =
  | 'auto_presented'
  | 'auto_received'
  | 'manually_approved'
  | 'manually_rejected'
  | 'queued'
  | 'rejected'
  | 'expired';

export interface AuditEvent {
  id: string;
  nodeId: string;
  requestId: string;
  action: AuditAction;
  linkType: LinkType;
  ruleId?: string;
  ruleLabel?: string;
  verifierDid?: string;
  issuerDid?: string;
  credentialType?: string;
  requestedFields?: string[];
  timestamp: string;
}
```

```typescript
// ==============================================================
// Discovery
// ==============================================================

export interface NodeCredentialTypeClaim {
  name: string;
  namespace?: string;
  displayName?: string;
}

export interface NodeCredentialType {
  id: string;         // e.g. "PhotoID"
  format: string;     // "mso_mdoc" | "dc+sd-jwt"
  displayName?: string;
  claims: NodeCredentialTypeClaim[];
}

// ==============================================================
// Intake response
// ==============================================================

export type IntakeAction = 'auto_executed' | 'queued' | 'rejected' |
'manual_required';

export interface IntakeResult {
  requestId: string;
  action: IntakeAction;
  linkType: LinkType;
  reason?: string;
  queuedItem?: PendingRequest;  // populated when action === 'queued'
}
```

Also update `src/types/index.ts` — extend `ViewName`:

typescript
```typescript
export type ViewName =
  | 'login'
  | 'dashboard'
```

```
  | 'detail'
  | 'receive'
  | 'present'
  | 'account'
  | 'consent_rules'
  | 'consent_rule_editor'
  | 'consent_queue'
  | 'consent_queue_detail'
  | 'audit_log';
```

SECTION 6 — CE CONTEXT (`src/context/ConsentEngineContext.tsx`)

Create a React context that manages CE connection state and provides CE data to screens.

6.1 State Shape

typescript

```
interface ConsentEngineState {
  ceUrl: string | null;                // Current CE base URL (null if not configured)
  ceEnabled: boolean;                  // User-level toggle
  ceApiKey: string | null;             // Raw API key (from localStorage; never null when configured)
  isConnected: boolean;                // Last health check was ok
  pendingCount: number;                // Number of items with status "pending" in the queue
  lastChecked: number | null;          // Timestamp of last health check
}
```

6.2 Context Functions

typescript

```
interface ConsentEngineContextValue {
  state: ConsentEngineState;
```

```
  configureCe: (ceUrl: string, apiKey: string) => Promise<void>;   // Save CE config, validate, prime module
  removeCe: () => void;                   // Clear CE config
  toggleCe: (enabled: boolean) => void;  // Enable/disable without changing config
  refreshHealth: () => Promise<void>;    // Manual health check + pending count refresh
  refreshPendingCount: () => Promise<void>; // Lightweight poll for badge count only
}
```

6.3 Behaviour

On mount / context initialisation:

1. Read `neoke_ce_url`, `neoke_ce_enabled`, `neoke_ce_apikey` from `localStorage`.
2. If CE URL exists and enabled: call `setCeBaseUrl()` on the CE client module and call `refreshHealth()`.
3. Set initial `isConnected` and `pendingCount` from health response.

`configureCe(ceUrl, apiKey)`:

1. Validate `ceUrl` reachability via `validateCeUrl()`.
2. Validate `apiKey` by calling `checkCeHealth()` with it — if the health endpoint returns `nodeConnected: false`, warn but do not block.
3. Store `ceUrl`, `apiKey`, and `enabled: true` in `localStorage`.
4. Update context state.
5. Call `setCeBaseUrl(ceUrl)`.

`refreshHealth()`:

1. Call `checkCeHealth()`.
2. Update `isConnected`, `pendingCount`, `lastChecked`.

Background polling:

- Poll `refreshPendingCount()` every 30 seconds when CE is configured and enabled.
- Use `setInterval` in a `useEffect` with the standard cleanup pattern.
- Do not poll when `ceEnabled` is false.

On logout (triggered by `AuthContext`):

- When `useAuth().state.token` becomes null, reset CE context state to defaults but do **not** clear `localStorage` CE config — the user may be re-authenticating to the same node.
- Clear CE config (`localStorage` + state) only when `useAuth().logout()` is called (full sign-out, not token expiry).

6.4 Provider Placement

Wrap the app in both providers:

tsx
```
<AuthProvider>
  <ConsentEngineProvider>
    <AppInner />
  </ConsentEngineProvider>
</AuthProvider>
```

SECTION 7 — MODIFIED `src/App.tsx`

7.1 New ViewName Handling

Add cases for the five new CE screens:

tsx
```
{currentView === 'consent_rules' && (
  <ConsentRulesScreen
    key="consent_rules"
    navigate={navigate}
  />
)}
{currentView === 'consent_rule_editor' && (
  <ConsentRuleEditorScreen
    key="consent_rule_editor"
    navigate={navigate}
```

```
    editingRuleId={editingRuleId}         // null = create, non-null =
edit
  />
)}
{currentView === 'consent_queue' && (
  <ConsentQueueScreen
    key="consent_queue"
    navigate={navigate}
  />
)}
{currentView === 'consent_queue_detail' && (
  <ConsentQueueDetailScreen
    key="consent_queue_detail"
    navigate={navigate}
    queueItemId={selectedQueueItemId}
  />
)}
{currentView === 'audit_log' && (
  <AuditLogScreen
    key="audit_log"
    navigate={navigate}
  />
)}
```

Add new state variables:

- `editingRuleId: string | null` — set when navigating to `consent_rule_editor` for editing.
- `selectedQueueItemId: string | null` — set when navigating to `consent_queue_detail`.

Extend `navigate()` to accept these in `extra`:

typescript
```
navigate(view: ViewName, extra?: {
  selectedCredential?: Credential;
  pendingUri?: string;
  editingRuleId?: string | null;
  selectedQueueItemId?: string | null;
```

```
})
```

7.2 Updated Tab Bar

The tab bar gains a fourth tab — "Consent" — when CE is configured and enabled:

tsx
```tsx
function TabBar({ currentView, onNavigate, ceEnabled, pendingCount })
{
  // Existing: Home, Scan, Account
  // New: Consent (only when ceEnabled)
  const consentActive =
['consent_rules','consent_queue','consent_queue_detail','audit_log','c
onsent_rule_editor'].includes(currentView);

  return (
    <div ...>
      {/* Home — unchanged */}
      {/* Scan QR Code — unchanged */}

      {/* Consent (conditional) */}
      {ceEnabled && (
        <button
          className={`flex-1 flex flex-col items-center gap-1 pt-3
pb-3 relative transition-colors ${consentActive ? 'text-[#5B4FE9]' :
'text-[#8e8e93]'}`}
          onClick={() => onNavigate('consent_queue')}
          aria-label="Consent"
        >
          {/* Shield icon */}
          <svg width="24" height="24" viewBox="0 0 24 24" fill="none">
            <path
              d="M12 2L4 6v6c0 5.25 3.5 10.15 8 11.35C16.5 22.15 20
17.25 20 12V6l-8-4z"
              stroke="currentColor"
              strokeWidth="1.7"
              fill={consentActive ? 'currentColor' : 'none'}
              fillOpacity={consentActive ? 0.12 : 0}
```

```
            />
          </svg>
          <span className="text-[10px] font-medium">Consent</span>
          {/* Badge for pending count */}
          {pendingCount > 0 && (
            <span className="absolute top-2 right-4 min-w-[18px]
h-[18px] bg-red-500 text-white text-[10px] font-bold rounded-full flex
items-center justify-center px-1">
              {pendingCount > 99 ? '99+' : pendingCount}
            </span>
          )}
        </button>
      )}

      {/* Account — unchanged */}
    </div>
  );
}
```

`showTabBar` should now include all consent-related views:

typescript
```
const showTabBar =
['dashboard','account','consent_rules','consent_queue','audit_log'].in
cludes(currentView);
```

7.3 Modified Deep-Link Routing

Currently, after authentication, a deep link navigates directly to `ReceiveScreen` or `PresentScreen`. With Layer 3, add CE routing **before** the existing direct routing:

typescript
```
// In the useEffect that fires on state.token change:
if (state.token && deepLinkUri && !deepLinkConsumed.current &&
deepLinkType !== 'unknown') {
  deepLinkConsumed.current = true;
  window.history.replaceState({}, '', window.location.pathname);
```

```
  if (ceEnabled && ceApiKey) {
    // Route through CE
    navigate('dashboard'); // show dashboard while CE processes
    setCeProcessingUri(deepLinkUri); // triggers CeIntakeOverlay (see
Section 8.7)
    return;
  }

  // Fallback: direct mode (existing behaviour)
  if (deepLinkType === 'receive') {
    navigate('receive', { pendingUri: deepLinkUri });
  } else {
    navigate('present', { pendingUri: deepLinkUri });
  }
}
```

Add `ceProcessingUri: string | null` state. When non-null, render `CeIntakeOverlay` (a full-screen overlay — see Section 8.7) on top of the current view.

SECTION 8 — NEW SCREENS AND COMPONENTS

8.1 `src/screens/AccountScreen.tsx` — Modifications

Add a new **"Consent Engine" section** at the bottom of the Account screen, above the Logout button.

When CE is not configured:

text

```
┌──────────────────────────────────────────────┐
│  Consent Engine                              │
│  Automate credential sharing rules           │
│  ┌──────────────────────────────────────────┐│
│  │  Not configured                          │
│  │  [Set up Consent Engine →]               │
│  └──────────────────────────────────────────┘│
```

```
└──────────────────────────────────────────────┘
```

Tapping "Set up Consent Engine →" opens an inline expansion (not a new screen) with two fields:

- **CE URL** — text input, placeholder `https://consent.example.com`
- **API Key** — password input (masked), pre-filled note: "Same API key you use to connect to this node"

And a "Connect" button. On tap:

1. Show loading spinner.
2. Call `configureCe(ceUrl, apiKey)` from context.
3. On success: collapse expansion, show connected state.
4. On error: show error message inline.

When CE is configured and connected:

text

```
┌──────────────────────────────────────────────┐
│  Consent Engine            ● Connected       │
│  consent.example.com                         │
│  ┌──────────────────────────────────────────┐│
│  │  Rules active: 3   Pending: 2            │
│  │  Last checked: 2 minutes ago             │
│  └──────────────────────────────────────────┘│
│  [Manage Rules]  [View Queue]  [Disconnect]  │
└──────────────────────────────────────────────┘
```

- "Manage Rules" → `navigate('consent_rules')`
- "View Queue" → `navigate('consent_queue')`
- "Disconnect" → show confirmation dialog → `removeCe()`

When CE is configured but disconnected:

Same layout but shows ⚠ `Disconnected` and a "Retry" button that calls `refreshHealth()`.

8.2 `src/screens/ConsentRulesScreen.tsx`

Purpose: List all consent rules. Allow creating, enabling/disabling, and deleting rules. Navigate to editor for creating/editing.

Layout:

text

```
← Back    Consent Rules              [+ New Rule]
──────────────────────────────────────────────────

Filter: [All ▾]  [Verification ▾]  [Issuance ▾]


┌──────────────────────────────────────────────┐
│ ● My airline check-in rule          Enabled  │
│ Verification · Any credential · Any party    │
│ Used 3 times · Never expires                 │
│                            [Edit] [Delete]   │
└──────────────────────────────────────────────┘


┌──────────────────────────────────────────────┐
│ ○ Age gate — nightclub              Disabled │
│ Verification · AgeVerification · *.bar.com   │
│ Used 0 times · Expires 2027-01-01            │
│                            [Edit] [Delete]   │
└──────────────────────────────────────────────┘


  [ + Create your first rule ]
  (shown when list is empty)
```

Interactions:

- **Toggle switch** on each rule card calls `enableRule()` or `disableRule()`.
- **Edit** → `navigate('consent_rule_editor', { editingRuleId: rule.id })`.
- **Delete** → confirmation dialog → `deleteRule()` → refresh list.
- **[+ New Rule]** button and **[+ Create your first rule]** empty state both navigate to `consent_rule_editor` with `editingRuleId: null`.

Data loading:

- Load on mount: `listRules(apiKey)`.
- Show loading skeleton (three placeholder cards) while loading.
- Show error banner with "Retry" on failure.

- Pull-to-refresh is not needed for a web app — provide a manual "Refresh" icon in the top-right corner area if desired.

Empty state:
When no rules exist, show a centred card:

text
🛡️
```
No consent rules yet.
Rules tell the Consent Engine which requests to
handle automatically on your behalf.
[Create your first rule]
```

8.3 `src/screens/ConsentRuleEditorScreen.tsx`

Purpose: Create a new rule or edit an existing one. This is a multi-step wizard, not a single long form, because the options are context-dependent.

Step 1 — Rule Type

text
```
What should this rule handle?

○  Verification requests
   (someone asking you to share credentials)

○  Credential offers
   (an issuer offering you a new credential)

[Cancel]                              [Next →]
```

Step 2 — Credential Type

text
```
Which credential type should this rule apply to?

○  Any credential type (catch-all)
```

```
○  Specific type:
   [Dropdown: populated from GET /node/{nodeId}/credentials]
      - Any
      - PhotoID (mso_mdoc)
      - AgeVerification (dc+sd-jwt)
      - MobileDriversLicense (mso_mdoc)
      - ...

[← Back]                              [Next →]
```

For verification rules only — Step 3 — Requesting Party:

text
```
Who is allowed to trigger this rule?

○  Anyone (any verifier)

○  Specific DID:
   [Text input: did:web:example.com]

○  Domain:
   [Text input: example.com]

○  Domain wildcard:
   [Text input: *.airline.com]

[← Back]                              [Next →]
```

For verification rules only — Step 4 — Allowed Fields:

text
```
Which fields may be shared?

○  Any fields the verifier asks for

○  Only specific fields:
```

```
   [Multi-select checklist — populated from the selected credential
type's
    claim list, fetched from GET /node/{nodeId}/credentials]
      ☑ Family Name
      ☑ Given Name
      ☐ Portrait
      ☐ Date of Birth
      ☐ Document Number

[← Back]                                [Next →]
```

If "Any credential type" was selected in Step 2, this step is skipped (no claims to filter).

For issuance rules only — Step 3 — Trusted Issuer:

text
```
Which issuers should this rule accept offers from?

○  Any issuer

○  Specific issuer DID:
   [Text input: did:web:issuer.example.com]

[← Back]                                [Next →]
```

For issuance rules with tx_code — Step 4 — Transaction Code (PIN):

text
```
Does this issuer require a PIN?

○  No PIN required

○  Yes, store PIN:
   [Masked text input — 4–8 characters]

Note: The PIN is stored securely and used automatically.
      Only set this if you trust this issuer fully.
```

```
[← Back]                                [Next →]
```

Step N — Conditions (optional):

text
```
Add conditions (optional)

These limit when the rule fires.

┌──────────────────────────────────────────────┐
│ + Add condition                              │
└──────────────────────────────────────────────┘

[Already added conditions shown here as chips]

○  Time of day: only between [08:00] and [20:00] UTC
○  Day of week: [Mon] [Tue] [Wed] [Thu] [Fri] (checkboxes)
○  Max per day: [10] times
○  Require linked domain: verifier must have a verified domain

[← Back]                                [Next →]
```

Step N+1 — Expiry:

text
```
When should this rule expire?

○  Never

○  On a specific date:
   [Date picker: ________]

○  After a number of uses:
   [Number input: ___] times

[← Back]                                [Next →]
```

Step N+2 — Label and Review:

text
```
Rule label (optional)

[Text input: e.g., "My airline check-in rule"]

──────── Review ──────────────────────────────
Type:         Verification
Credential:   AgeVerification (dc+sd-jwt)
Party:        *.airline.com
Fields:       Any
Conditions:   Weekdays only, 08:00–20:00 UTC
Expiry:       After 50 uses

[← Back]                      [Save Rule ✓]
```

On save:

- Call `createRule(apiKey, payload)` (create) or `updateRule(apiKey, id, payload)` (edit).
- Show loading spinner on the button.
- On success: navigate back to `consent_rules`.
- On error: show inline error below the button; do not navigate away.

8.4 `src/screens/ConsentQueueScreen.tsx`

Purpose: Show all pending requests that the CE queued because no rule matched. Allow the user to approve or reject each one.

Layout:

text
```
← Back          Pending Approval           [All ▾]

  2 requests waiting for your decision

┌──────────────────────────────────────────────┐
```

```
│ 🔒 Verification Request                    │
│ ServiceX                    2 min ago      │
│ servicex.com                               │
│ Asking for: Family Name, Date of Birth     │
│ Credential: PhotoID                        │
│ Expires in: 23h 45m                        │
│                    [Reject]  [Approve →]   │
└────────────────────────────────────────────┘
```

```
┌────────────────────────────────────────────┐
│ 📄 Credential Offer                        │
│ did:web:gov-issuer.example.com  5 min ago  │
│ Offering: MobileDriversLicense             │
│ Requires PIN: No                           │
│ Expires in: 23h 40m                        │
│                    [Reject]  [Accept →]    │
└────────────────────────────────────────────┘
```

Colour coding:

- Verification request card: left border purple (`#5B4FE9`)
- Credential offer card: left border green (`#059669`)
- Expired items: greyed out, no action buttons

Quick actions from list:

- **Reject** button (small, secondary) on each card: tap → confirmation popover → `rejectQueueItem()` → remove from list.
- **Approve** → / **Accept** → button: tap → navigate to `consent_queue_detail` with the item ID.

Filter tabs:

- All / Pending / Approved / Rejected

Empty state:

text

```
✓ All caught up
No pending requests.
```

Expiry warning:
If a pending request will expire within 1 hour, show an orange "Expiring soon" badge on the card.

8.5 `src/screens/ConsentQueueDetailScreen.tsx`

Purpose: Full detail view of a single pending request. Shows all available context to help the user make an informed decision.

Layout — Verification Request:

text
```
← Pending          Verification Request

┌──────────────────────────────────────────┐
│  Who is asking?                          │
│  ServiceX                                │
│  servicex.com                            │
│  DID: did:web:servicex.com               │
│  ✓ Linked domain verified                │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│  What they're asking for                 │
│  PhotoID (mso_mdoc)                      │
│  • Family Name                           │
│  • Date of Birth                         │
│                        (2 of 5 fields)   │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│  From which credential?                  │
│  [Card: PhotoID · Qatar Issuer]          │
│  Issued: Jan 2024 · Expires: Jan 2029    │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│  ⚠ Expires in 23h 45m                    │
```

```
│  After expiry this request will be          │
│  automatically discarded.                   │
└─────────────────────────────────────────────┘

─────────────────────────────────────────────

[Also create a rule for future requests]   ←toggle

─────────────────────────────────────────────

        [Reject]           [Share Now →]
```

Layout — Credential Offer:

text

```
← Pending          Credential Offer

┌─────────────────────────────────────────────┐
│  Who is offering?                           │
│  did:web:gov-issuer.example.com             │
└─────────────────────────────────────────────┘

┌─────────────────────────────────────────────┐
│  What they're offering                      │
│  MobileDriversLicense (mso_mdoc)            │
│  Includes: Family Name, Given Name,         │
│  Date of Birth, Driving Privileges,         │
│  Document Number, Expiry Date               │
└─────────────────────────────────────────────┘

┌─────────────────────────────────────────────┐
│  PIN required? No                           │
└─────────────────────────────────────────────┘

─────────────────────────────────────────────

        [Reject]          [Accept Credential →]
```

"Also create a rule" toggle (verification only):
When toggled on, show an inline mini-form:

text
```
Create rule for:
  ○ This exact verifier (DID)
  ○ This verifier's domain
  ○ Anyone

Fields to allow:
  ○ Only the fields in this request
  ○ Any fields

[Rule will be created when you tap Share Now]
```

On approval with rule creation enabled:

1. Call `approveQueueItem(apiKey, id)`.
2. On success, call `createRule(apiKey, rulePayload)` with the constructed rule.
3. Show success overlay (see Section 8.6).

On approval (no rule creation):

- Show loading spinner on button.
- For credential offers with PIN required: show a PIN entry sheet before proceeding.
- Call `approveQueueItem(apiKey, id, txCode?)`.
- On success: show success overlay, navigate back to queue.
- On error: show inline error.

On rejection:

- Show confirmation bottom sheet: "Are you sure you want to reject this request? This cannot be undone."
- Call `rejectQueueItem(apiKey, id)`.
- Navigate back to queue.

8.6 `src/screens/AuditLogScreen.tsx`

Purpose: Read-only history of all CE decisions.

Layout:

text

```
← Back                Activity Log

Filter: [All ▾]  [Automated ▾]  [Manual ▾]

  March 10, 2026

┌──────────────────────────────────────────┐
│ ✓ Auto-shared          12:45 PM          │
│ ServiceX · Family Name, Date of Birth    │
│ Rule: "My airline check-in rule"         │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│ ✓ Credential received  11:30 AM          │
│ gov-issuer · MobileDriversLicense        │
│ Rule: "Accept all gov credentials"       │
└──────────────────────────────────────────┘

  March 9, 2026

┌──────────────────────────────────────────┐
│ ✗ Rejected              5:12 PM          │
│ unknown-service.com · AgeVerification    │
│ Manually rejected by user               │
└──────────────────────────────────────────┘
```

Action icons/colours:

- `auto_presented`, `auto_received`, `manually_approved`: green ✓
- `manually_rejected`, `rejected`: red ✗
- `queued`: yellow ⏳
- `expired`: grey ⌛

Infinite scroll / pagination:

- Load first 20 events on mount.

- When the user scrolls near the bottom, load the next 20 (`offset` parameter).
- Show loading spinner at bottom during fetch.

Filter by action type:

- "All" — no filter
- "Automated" — `auto_presented`, `auto_received`
- "Manual" — `manually_approved`, `manually_rejected`
- "Rejected/Expired" — `rejected`, `expired`

Empty state:

text
```
No activity yet.
Events will appear here as the Consent Engine
handles requests on your behalf.
```

8.7 `src/components/CeIntakeOverlay.tsx`

Purpose: A full-screen overlay shown over the current view while the CE processes a deep link. This replaces the direct navigation to `ReceiveScreen` / `PresentScreen` when CE is configured.

States and layout:

Processing:

text
```
┌──────────────────────────────────────────┐
│                                          │
│          [Animated shield spinner]       │
│                                          │
│    Checking your consent rules...        │
│                                          │
└──────────────────────────────────────────┘
```

auto_executed — success:

text

```
┌──────────────────────────────────────────┐
│                                          │
│          ✓  (large, green, animated)     │
│                                          │
│         Handled automatically            │
│                                          │
│  Your consent rules approved and         │
│  processed this request automatically.   │
│                                          │
│           [View Activity →]   [Close]    │
└──────────────────────────────────────────┘
```

Auto-dismiss after 3 seconds if the user does not tap.

queued — waiting:

text

```
┌──────────────────────────────────────────┐
│                                          │
│          ⏳  (large, yellow)             │
│                                          │
│        Waiting for your approval         │
│                                          │
│  No rule matched this request. It has    │
│  been added to your approval queue.      │
│                                          │
│      [Review Now →]           [Later]    │
└──────────────────────────────────────────┘
```

"Review Now →" navigates to `consent_queue_detail` for the new item.

rejected:

text

```
┌──────────────────────────────────────────┐
│                                          │
│          ✗  (large, red)                 │
│                                          │
│            Request declined              │
```

```
│                                                │
│  [reason from CE response]                    │
│                                                │
│                          [Close]               │
└────────────────────────────────────────────────┘
```

manual_required — fallback to direct flow:

Do not show an overlay. Immediately dismiss and fall through to the existing `ReceiveScreen` / `PresentScreen` with the original URI. This is the graceful fallback for edge cases.

Error (CE unreachable):

text

```
┌────────────────────────────────────────────────┐
│          ⚠ Consent Engine unavailable          │
│                                                │
│  Could not reach the Consent Engine.          │
│  Process this request manually instead?      │
│                                                │
│   [Cancel]          [Handle Manually →]       │
└────────────────────────────────────────────────┘
```

"Handle Manually →" falls through to direct flow.

Implementation:

tsx

```tsx
// App.tsx — render overlay above all screens when ceProcessingUri is set
{ceProcessingUri && (
  <CeIntakeOverlay
    rawLink={ceProcessingUri}
    onDismiss={() => setCeProcessingUri(null)}
    onFallback={(uri, type) => {
      setCeProcessingUri(null);
      if (type === 'receive') navigate('receive', { pendingUri: uri });
      else navigate('present', { pendingUri: uri });
```

```
    }}
    onReviewQueue={(itemId) => {
      setCeProcessingUri(null);
      navigate('consent_queue_detail', { selectedQueueItemId: itemId
});
    }}
    onViewAudit={() => {
      setCeProcessingUri(null);
      navigate('audit_log');
    }}
  />
)}
```

The overlay calls `ceIntake(apiKey, rawLink)` on mount, then transitions through states based on the response.

8.8 `src/components/CeStatusBanner.tsx` (Optional, Recommended)

A small, non-intrusive banner shown at the top of `DashboardScreen` when:

- CE is configured but disconnected: `"⚠ Consent Engine offline · Tap to retry"`
- CE has pending items: `"🔔 {n} request{s} awaiting your approval · Review →"` — tapping navigates to `consent_queue`

Do not show when CE is not configured or when everything is healthy and no pending items.

SECTION 9 — MODIFIED RECEIVE AND PRESENT SCREENS

The existing `ReceiveScreen` and `PresentScreen` do not need major changes. Their direct flow is the **fallback** path used when:

- CE is not configured.

- CE returns `manual_required`.
- CE is unreachable and user chooses "Handle Manually".

One modification needed in each:

Add a small informational note at the top when the wallet is operating in direct mode but CE is configured:

text
```
ℹ️  Consent Engine bypassed for this request.
```

This helps users understand why CE automation did not fire (e.g., the link was scanned via QR camera manually, not routed through CE).

Alternatively, when the user scans a QR code directly in `ReceiveScreen` / `PresentScreen` (not via a deep link), the wallet should optionally route through CE before proceeding. Implement this as follows:

In `ReceiveScreen` and `PresentScreen`, after the URI is obtained (either from `initialUri` or from a manual QR scan), check:

typescript
```typescript
if (isCeConfigured() && ceEnabled && ceApiKey) {
  // Route through CE instead of direct processing
  setCeProcessingUri(uri); // passed via a callback prop or lifted state
  return;
}
// Otherwise continue with direct flow as today
```

This ensures that **all link handling**, whether from deep links or manual QR scans, goes through the CE when it is configured.

Continuing from Section 10 exactly where it was cut off.

SECTION 10 — CONSENT ENGINE CONFIGURATION IN ONBOARDING

The onboarding flow currently has two steps: Step 1 (node identifier) and Step 2 (API key). Layer 3 adds an optional Step 3 that appears after successful authentication.

10.1 When Step 3 Appears

Step 3 is shown **once** after a fresh login if:

- No CE URL is already stored in `localStorage` (first-time setup), **or**
- The stored CE URL is no longer reachable (detected during `refreshHealth()` on startup).

Step 3 is **skipped** if:

- A valid CE URL is already stored and healthy.
- The user has previously dismissed Step 3 and set a `neoke_ce_dismissed` flag in `localStorage`.

10.2 OnboardingStep3Screen Layout

text

```
┌──────────────────────────────────────────┐
│                                          │
│            🛡️  Automate Consent          │
│                                          │
│  Connect a Consent Engine to automatically │
│  handle credential requests using rules  │
│  you define — without being present.     │
│                                          │
│  Consent Engine URL                      │
│  ┌──────────────────────────────────┐    │
│  │ https://consent.example.com      │    │
│  └──────────────────────────────────┘    │
│                                          │
│  Your API Key will be used to connect.   │
│  (same key you just entered)             │
│                                          │
│        [Skip for now]   [Connect →]      │
│                                          │
│  You can set this up later in Account.   │
└──────────────────────────────────────────┘
```

On "Connect →":

1. Validate the CE URL (must be `https://`, must be reachable via `validateCeUrl()`).
2. Call `configureCe(ceUrl, apiKey)` — the `apiKey` is available in-memory from Step 2 at this point. Pass it directly; do not ask again.
3. On success: navigate to `dashboard`.
4. On error: show inline error, stay on Step 3.

On "Skip for now":

- Set `neoke_ce_dismissed: "true"` in `localStorage`.
- Navigate to `dashboard`.

The `neoke_ce_dismissed` flag prevents Step 3 from reappearing on every login. It is cleared when the user logs out (so a new login session can prompt again, e.g., after a node change).

SECTION 11 — NAVIGATION AND SCREEN MAP

Here is the complete navigation map for Layer 3. Every screen knows which screens it can navigate to.

text
```
App (App.tsx)
│
├── OnboardingStep1Screen
│   └── → OnboardingStep2Screen
│       └── → OnboardingStep3Screen (optional)
│           └── → DashboardScreen
│
├── DashboardScreen
│   ├── → CredentialDetailScreen (tap a credential card)
│   └── → ConsentQueueScreen (via CeStatusBanner)
│
├── CredentialDetailScreen
│   └── → DashboardScreen (back)
│
├── ReceiveScreen (scan or deep link — direct mode)
│   └── → DashboardScreen (done / cancel)
│
```

```
├── PresentScreen (scan or deep link — direct mode)
│   └── → DashboardScreen (done / cancel)
│
├── AccountScreen
│   ├── → ConsentRulesScreen (Manage Rules)
│   ├── → ConsentQueueScreen (View Queue)
│   └── → OnboardingStep1Screen (logout)
│
├── ConsentRulesScreen
│   ├── → ConsentRuleEditorScreen (new rule or edit)
│   └── → AccountScreen (back)
│
├── ConsentRuleEditorScreen
│   └── → ConsentRulesScreen (save or cancel)
│
├── ConsentQueueScreen
│   ├── → ConsentQueueDetailScreen (tap item)
│   └── → AccountScreen (back via tab)
│
├── ConsentQueueDetailScreen
│   ├── → ConsentQueueScreen (back, approve, reject)
│   └── → ConsentRuleEditorScreen (create rule from approval —
pre-filled)
│
└── AuditLogScreen
    └── → AccountScreen (back via tab)
```

Tab bar navigation (when CE is enabled):

- Home tab → `dashboard`
- Scan tab → `receive`
- Consent tab → `consent_queue`
- Account tab → `account`

Tab bar navigation (when CE is not configured):

- Home tab → `dashboard`
- Scan tab → `receive`
- Account tab → `account`

The Consent tab is only rendered when `ceEnabled === true` in `ConsentEngineContext`. Its badge shows `pendingCount` from context state. When `pendingCount === 0`, the badge is hidden.

SECTION 12 — CE CLIENT INITIALISATION AND LIFECYCLE

12.1 Startup Sequence

On app mount:

text
```
1. AuthProvider initialises
   └── Reads neoke_s_token, neoke_s_expires, neoke_node_id from
localStorage
   └── If token present and not expired: restores session (sets
baseUrl, token)
   └── If token expired: sets sessionExpired flag (ReAuthModal
appears)
   └── If no token: shows onboarding

2. ConsentEngineProvider initialises (after AuthProvider)
   └── Reads neoke_ce_url, neoke_ce_enabled, neoke_ce_apikey from
localStorage
   └── If CE URL present and enabled:
       └── Calls setCeBaseUrl(ceUrl)
       └── Calls refreshHealth()
           └── Updates isConnected, pendingCount
   └── Starts background poll (every 30 seconds)
       └── Calls refreshPendingCount() → GET /health
       └── Updates pendingCount in state (triggers badge re-render)
```

12.2 Token vs API Key

The IDN Node uses a bearer **token** for all calls (short-lived, ~1 hour). The CE uses an **API key** for all calls (the same raw API key entered at onboarding, stored in `localStorage` under `neoke_ce_apikey`).

This is intentional: the CE is a separate service and does not share the node's token lifecycle. The API key does not expire. The CE validates it by making its own auth call to the node.

Implication for the wallet:

- When calling any CE API, always pass the `apiKey` from `ConsentEngineContext` state.
- If `ceApiKey` is null (not stored), CE calls cannot be made. Show a "reconnect" prompt in AccountScreen.
- Never pass the bearer `token` (from `AuthContext`) to the CE. These are different credentials for different services.

12.3 Handling CE Errors

text

```
CE unreachable (network error)
└── Update isConnected: false in context
└── Show CeStatusBanner "⚠ Consent Engine offline"
└── Fall back to direct mode for any new link

CE returns 401
└── API key is invalid or revoked
└── Show AccountScreen prompt: "Consent Engine API key rejected — reconnect"
└── Clear neoke_ce_apikey from localStorage
└── Set isConnected: false

CE returns 503 (degraded)
└── isConnected: false
└── Log warning, continue with degraded banner
└── Do not fall back to direct mode for already-queued items — they stay in queue
```

SECTION 13 — DATA FLOWS: FULL WALKTHROUGHS

13.1 User Opens a Deep Link — CE Configured

text

```
User opens: https://wallet.example.com/?uri=openid4vp%3A%2F%2F...

App.tsx: on mount
└── Extracts deepLinkUri from window.location.search
└── deepLinkType = "present"

AuthContext: state.token = "abc..." (already logged in)

useEffect fires:
└── deepLinkUri is present, not consumed
└── ceEnabled = true, ceApiKey = "key123"
└── Sets ceProcessingUri = deepLinkUri
└── Navigates to "dashboard"
└── Marks deepLinkConsumed = true
└── Cleans URL: window.history.replaceState({}, '', '/')

CeIntakeOverlay renders:
└── Calls ceIntake("key123", "openid4vp://...")
└── Shows "Checking your consent rules..."

    POST /consent/intake
    { "rawLink": "openid4vp://..." }
    Authorization: ApiKey key123
    → CE processes: preview → rule match → respond
    ← { action: "auto_executed", requestId: "uuid1", linkType:
"vp_request" }

CeIntakeOverlay transitions to "auto_executed" state:
└── Shows ✓ "Handled automatically"
└── Auto-dismisses after 3 seconds
└── Or user taps "View Activity →" → navigates to audit_log

Background poll (30s):
└── GET /health → pendingCount: 0
└── Badge stays hidden
```

13.2 User Scans QR Code — CE Configured — No Rule Matches

text

```text
User is on DashboardScreen, taps "Scan QR Code" tab

ReceiveScreen or PresentScreen opens
User scans a QR code containing: openid-credential-offer://...

Screen parses the raw URI
└── isCeConfigured() = true, ceEnabled = true, ceApiKey = "key123"
└── Calls onRouteToCe(uri) — a prop callback passed from App.tsx
└── App.tsx sets ceProcessingUri = uri, stays on current screen

CeIntakeOverlay renders:
└── Calls ceIntake("key123", "openid-credential-offer://...")
└── Shows "Checking your consent rules..."

    POST /consent/intake
    { "rawLink": "openid-credential-offer://..." }
    → CE processes: fetch offer → no rule matches
    ← { action: "queued", requestId: "uuid2", linkType:
"credential_offer",
        queuedItem: { id: "uuid2", preview: { credentialTypes:
["MobileDriversLicense"], ... } } }

CeIntakeOverlay transitions to "queued" state:
└── Shows ⏳ "Waiting for your approval"
└── Shows "No rule matched — added to queue"
└── "Review Now →" button navigates to consent_queue_detail/uuid2
└── "Later" dismisses overlay

Background poll (next 30s):
└── GET /health → pendingCount: 1
└── Badge shows "1" on Consent tab
```

13.3 User Approves a Queue Item and Creates a Rule

text

```
User taps Consent tab → ConsentQueueScreen

GET /queue?status=pending
└── Returns 1 item: credential offer for MobileDriversLicense from
did:web:gov-issuer.com

User taps "Accept →"

ConsentQueueDetailScreen loads:
└── GET /queue/uuid2
└── Shows: issuer, credential type, fields, PIN required: No

User toggles "Also create a rule for future requests"
└── Selects: "This issuer's DID" + "This credential type"
└── Mini rule preview shows: "Accept MobileDriversLicense from
did:web:gov-issuer.com"

User taps "Accept Credential →"

Step 1: approve queue item
    POST /queue/uuid2/approve
    Authorization: ApiKey key123
    ← { status: "approved" }

Step 2: create rule
    POST /rules
    Authorization: ApiKey key123
    Body: {
      ruleType: "issuance",
      credentialType: "MobileDriversLicense",
      party: { matchType: "any" },
      trustedIssuerDid: "did:web:gov-issuer.com",
      allowedFields: { mode: "any" },
      expiry: { type: "never" },
      label: "Accept MobileDriversLicense from gov-issuer"
    }
    ← { id: "rule-uuid3", ... }
```

```
ConsentQueueDetailScreen shows success overlay:
└── ✓ "Credential accepted"
└── "Rule created for future offers"
└── Auto-navigates back to ConsentQueueScreen after 2 seconds

ConsentQueueScreen now shows empty state: "✓ All caught up"

Background poll: pendingCount = 0 → badge hidden
```

13.4 User Creates a Rule from Scratch

text

```
User taps Account tab → AccountScreen
Taps "Manage Rules" → ConsentRulesScreen
Taps "+ New Rule" → ConsentRuleEditorScreen

Step 1: Verification
Step 2: Specific type → AgeVerification (dc+sd-jwt)
Step 3: Domain wildcard → *.bar.com
Step 4: Explicit fields → over_18 only
Step 5: Conditions → Day of week: Mon-Sun (all), Time of day:
09:00–23:00
Step 6: Expiry → Never
Step 7: Label → "Age gate — any bar"
         Review shown, user taps Save

    POST /rules
    Authorization: ApiKey key123
    Body: {
      ruleType: "verification",
      credentialType: "AgeVerification",
      credentialFormat: "dc+sd-jwt",
      party: { matchType: "domain_wildcard", value: "*.bar.com" },
      allowedFields: { mode: "explicit", explicit: ["over_18"] },
      conditions: [
        { type: "time_of_day", startHour: 9, endHour: 23 }
      ],
      expiry: { type: "never" },
```

```
      label: "Age gate — any bar"
    }
    ← { id: "rule-uuid4", enabled: true, ... }

Navigates back to ConsentRulesScreen
└── New rule "Age gate — any bar" appears in list, enabled
```

SECTION 14 — DIRECTORY STRUCTURE CHANGES

Only the **additions and modifications** are listed. Everything else in the existing project is unchanged.

text

```
src/
│
├── api/
│   ├── client.ts                  ← UNCHANGED (existing node API
client)
│   └── consentEngineClient.ts     ← NEW: CE HTTP API client
│
├── context/
│   ├── AuthContext.tsx            ← MODIFIED: store/clear
neoke_ce_apikey on login/logout
│   └── ConsentEngineContext.tsx   ← NEW: CE config, health, pending
count
│
├── screens/
│   ├── DashboardScreen.tsx        ← MODIFIED: add CeStatusBanner
│   ├── ReceiveScreen.tsx          ← MODIFIED: route through CE if
configured
│   ├── PresentScreen.tsx          ← MODIFIED: route through CE if
configured
│   ├── AccountScreen.tsx          ← MODIFIED: add CE config section
│   ├── OnboardingStep2Screen.tsx  ← MODIFIED: store apiKey in
localStorage at success
│   ├── OnboardingStep3Screen.tsx  ← NEW: optional CE setup step
│   ├── ConsentRulesScreen.tsx     ← NEW
```

```
|    ├── ConsentRuleEditorScreen.tsx← NEW (multi-step wizard)
|    ├── ConsentQueueScreen.tsx     ← NEW
|    ├── ConsentQueueDetailScreen.tsx ← NEW
|    └── AuditLogScreen.tsx         ← NEW
|
├── components/
|    ├── CeIntakeOverlay.tsx        ← NEW: full-screen CE processing
overlay
|    └── CeStatusBanner.tsx         ← NEW: dashboard status/pending
count banner
|
├── types/
|    ├── index.ts                   ← MODIFIED: extend ViewName
|    └── consentEngine.ts           ← NEW: all CE TypeScript types
|
└── App.tsx                         ← MODIFIED: new views, tab bar, CE
overlay state
```

SECTION 15 — UX DESIGN PRINCIPLES

Follow these principles consistently across all new screens. They are derived from the existing wallet's design language.

15.1 Visual Language

- **Background:** `#F2F2F7` (iOS system grouped background) — used on all screen backgrounds.
- **Cards:** `bg-white rounded-2xl shadow-sm` — white cards with subtle shadow.
- **Primary colour:** `#5B4FE9` (purple) — used for active states, primary buttons, accent borders.
- **Positive/green:** `#059669` — used for accepted/approved/received states.
- **Warning/orange:** `#F59E0B` — used for pending/expiring-soon states.
- **Danger/red:** `#EF4444` — used for rejected/revoked/expired states.
- **Body text:** `text-[#1C1C1E]` — near-black.
- **Secondary text:** `text-[#8e8e93]` — medium grey.

15.2 Animation

All existing screens use `framer-motion` with `AnimatePresence mode="wait"`. New screens must follow the same pattern:

tsx
```
import { motion } from 'framer-motion';

const variants = {
  initial: { opacity: 0, y: 16 },
  animate: { opacity: 1, y: 0, transition: { duration: 0.22, ease:
'easeOut' } },
  exit:    { opacity: 0, y: -8, transition: { duration: 0.14 } },
};

export default function ConsentRulesScreen() {
  return (
    <motion.div {...variants} className="flex flex-col min-h-screen
bg-[#F2F2F7]">
      ...
    </motion.div>
  );
}
```

15.3 Loading States

Always show loading skeletons (not a spinner alone) for list screens. Use placeholder card-shaped divs with a pulse animation:

tsx
```
<div className="animate-pulse bg-white rounded-2xl h-24 w-full" />
```

Show 2–3 skeletons while data loads.

15.4 Error States

All errors must be human-readable. Never show raw error codes or stack traces to the user. Map CE error codes to friendly messages:

CE error code	User-facing message
`RULE_NOT_FOUND`	"This rule no longer exists."
`REQUEST_NOT_FOUND`	"This request is no longer available."
`REQUEST_ALREADY_RESOLVED`	"This request has already been handled."
`REQUEST_EXPIRED`	"This request expired before it could be handled."
`NODE_DISCONNECTED`	"The Consent Engine cannot reach your wallet node right now."
`INVALID_API_KEY`	"Your API key was not accepted. Please reconnect in Account settings."
(generic 5xx)	"Something went wrong. Please try again."

15.5 Confirmation Dialogs

Use a bottom sheet pattern (not browser `confirm()`) for all destructive actions (delete rule, reject request). The bottom sheet has:

- Brief title: "Delete this rule?"
- Short consequence: "Rules you delete cannot be recovered."
- Two buttons: **Cancel** (secondary) and **Delete** / **Reject** (red, primary).

15.6 Safe Areas and Tab Bar

All screens must add `pb-24` bottom padding on screens that have the tab bar visible, to prevent content from being hidden behind it. This is consistent with the existing screens.

15.7 Touch Targets

All interactive elements (buttons, toggles, list items) must have a minimum touch target of `min-h-[44px]` — consistent with Apple HIG guidelines, which the existing wallet follows.

SECTION 16 — MODIFIED `AuthContext.tsx` CHANGES (Spec)

Two specific changes are needed. Do not rewrite the file — only add the following:

16.1 Store API Key at Login

In the `SET_TOKEN` reducer or at the point where login succeeds (in `OnboardingStep2Screen` after `apiKeyAuth()` returns), store the raw API key:

typescript
```
// In OnboardingStep2Screen.tsx, after successful apiKeyAuth():
try {
  localStorage.setItem('neoke_ce_apikey', apiKey);
} catch {
  // ignore storage errors
}
```

This ensures the CE API key is available immediately after onboarding Step 2, regardless of whether Step 3 runs.

16.2 Clear CE Config on Logout

In `AuthContext`'s `logout()` action handler, add:

typescript
```
try {
  localStorage.removeItem('neoke_ce_url');
  localStorage.removeItem('neoke_ce_enabled');
  localStorage.removeItem('neoke_ce_apikey');
  localStorage.removeItem('neoke_ce_dismissed');
} catch {
  // ignore
}
```

This ensures CE configuration does not persist across node changes.

SECTION 17 — TESTING STRATEGY

17.1 Unit Tests

`src/api/consentEngineClient.ts`:

- `setCeBaseUrl` / `getCeBaseUrl` / `isCeConfigured` work correctly.
- `ceIntake()` with `action: "auto_executed"` returns correct shape.
- `ceIntake()` with `action: "queued"` includes `queuedItem`.
- `ceIntake()` with CE unreachable throws `CeApiError` with connectivity message.
- `listRules()` with empty response returns `[]`.
- `approveQueueItem()` with `txCode` includes it in request body.
- `checkCeHealth()` maps `nodeConnected: false` correctly without throwing.

`src/context/ConsentEngineContext.tsx`:

- `configureCe()` saves URL, apiKey, enabled to localStorage.
- `removeCe()` clears all three localStorage keys.
- `toggleCe(false)` updates `ceEnabled` to false and stops polling.
- `refreshHealth()` updates `pendingCount` from health response.

`src/utils/uriRouter.ts` (existing, no changes):

- Ensure all existing tests still pass — do not break existing `detectUriType` behaviour.

`src/components/CeIntakeOverlay.tsx`:

- Renders "Checking..." on mount.
- Transitions to auto_executed state on success response.
- Transitions to queued state when action is "queued".
- Calls `onFallback()` when action is "manual_required".
- Calls `onFallback()` when CE throws connectivity error and user taps "Handle Manually".
- Auto-dismisses after 3 seconds on "auto_executed".

Rule editor wizard (multi-step):

- Step 1 selection of "verification" shows Steps 2, 3, 4 in correct order.
- Step 1 selection of "issuance" shows Steps 2, 3 (issuer), 4 (PIN) in correct order.
- Selecting "any credential type" in Step 2 skips the explicit fields step.
- Back button always goes to the previous step, preserving entered values.
- "Save Rule" button is disabled while saving is in progress.

- On save error, button re-enables and error message is shown.

17.2 Integration Tests (Using MSW)

Use the same `msw` approach specified in the Layer-2 PRD. Mock the CE API at `https://consent.test.com`.

Test: deep link with CE configured, auto_executed:

text
```
Setup: ConsentEngineContext with valid CE URL + apiKey
Mock: POST /consent/intake → { action: "auto_executed", requestId:
"r1", linkType: "vp_request" }
Action: simulate app mount with ?uri=openid4vp://... in
window.location.search
Assert: CeIntakeOverlay renders → shows "Handled automatically"
Assert: direct navigate to ReceiveScreen/PresentScreen does NOT happen
Assert: after 3s (or button tap), overlay dismisses
```

Test: deep link with CE configured, queued:

text
```
Mock: POST /consent/intake → { action: "queued", requestId: "r2",
queuedItem: { id: "r2", ... } }
Action: app mount with VP request URI
Assert: CeIntakeOverlay shows "Waiting for your approval"
Assert: "Review Now →" button visible
Assert: tap "Review Now →" → ConsentQueueDetailScreen shown for item
r2
```

Test: deep link with CE unreachable, user chooses manual:

text
```
Mock: POST /consent/intake → network error (no handler)
Action: app mount with VP request URI
Assert: CeIntakeOverlay shows "Consent Engine unavailable"
Assert: tap "Handle Manually →" → PresentScreen shown with original
URI as initialUri
```

Test: deep link with CE configured but disabled:

text
```
Setup: ceEnabled = false
Action: app mount with VP request URI
Assert: CeIntakeOverlay NOT rendered
Assert: direct navigation to PresentScreen with URI
```

Test: list rules — empty:

text
```
Mock: GET /rules → { rules: [] }
Action: navigate to ConsentRulesScreen
Assert: empty state "No consent rules yet" shown
Assert: "+ Create your first rule" button visible
```

Test: create rule — success:

text
```
Mock: GET /node/b2b-poc/credentials → { types: [{ id:
"AgeVerification", ... }] }
Mock: POST /rules → { id: "rule1", enabled: true, ... }
Action: complete all wizard steps with valid data, tap Save
Assert: POST /rules called with correct payload
Assert: navigate to ConsentRulesScreen after save
Assert: new rule appears in list
```

Test: create rule — CE returns 400:

text
```
Mock: POST /rules → 400 { error: { code: "INVALID_PAYLOAD", message:
"..." } }
Action: complete wizard, tap Save
Assert: error message shown below Save button
Assert: wizard stays on Review step (no navigation)
Assert: Save button re-enables
```

Test: approve queue item + create rule:

text
```
Mock: GET /queue/r1 → { id: "r1", linkType: "vp_request", status:
"pending", preview: {...} }
Mock: POST /queue/r1/approve → { status: "approved" }
Mock: POST /rules → { id: "rule2", ... }
Action: open ConsentQueueDetailScreen for r1
        Toggle "create rule", select "this verifier's DID"
        Tap "Share Now →"
Assert: POST /queue/r1/approve called
Assert: POST /rules called with verifier DID
Assert: success overlay shown
Assert: navigate back to ConsentQueueScreen
```

Test: reject queue item:

text
```
Mock: POST /queue/r1/reject → { status: "rejected" }
Action: tap "Reject" on queue item, confirm in bottom sheet
Assert: POST /queue/r1/reject called
Assert: item removed from list
Assert: badge count decrements
```

Test: account screen CE setup:

text
```
Mock: GET /health → { status: "ok", nodeConnected: true, pendingCount:
3 }
Action: enter CE URL + API key, tap Connect
Assert: validateCeUrl called, then checkCeHealth called
Assert: CE status shows "● Connected"
Assert: "Rules active: X  Pending: 3" shown
Assert: Consent tab appears in bottom tab bar with badge "3"
```

SECTION 18 — IMPLEMENTATION ORDER (MVP)

Implement in this exact order. Each phase is independently testable.

Phase 1 — Types and CE API Client (no UI yet)

1. Create `src/types/consentEngine.ts` with all CE types.
2. Update `src/types/index.ts` — extend `ViewName`.
3. Create `src/api/consentEngineClient.ts` — implement all exported functions with proper error handling.
4. Write unit tests for the CE client using msw mocks.

Phase 2 — CE Context (no UI yet)

5. Create `src/context/ConsentEngineContext.tsx` — full implementation including localStorage restore, health poll, `configureCe`, `removeCe`, `toggleCe`.
6. Wrap `App.tsx` with `ConsentEngineProvider` (no visible change yet).
7. Modify `AuthContext.tsx` — store API key at login, clear CE config at logout.
8. Write unit tests for context state transitions.

Phase 3 — Account Screen CE Section

9. Modify `AccountScreen.tsx` — add CE section (not configured / connected / disconnected states).
10. Manually test: enter a CE URL, verify health check, see connected status.
11. Test "Disconnect" flow — verify localStorage cleared.

Phase 4 — Onboarding Step 3

12. Create `OnboardingStep3Screen.tsx`.
13. Wire into `App.tsx` onboarding flow (after Step 2 success).
14. Test skip and connect paths.
15. Verify `neoke_ce_dismissed` flag prevents re-showing.

Phase 5 — Tab Bar and Navigation

16. Modify `TabBar` in `App.tsx` — add conditional Consent tab with badge.
17. Add new state variables to `App.tsx` (`editingRuleId`, `selectedQueueItemId`, `ceProcessingUri`).
18. Extend `navigate()` to pass new extras.
19. Add all new `currentView` cases to the main render block (stub screens for now — just `<div>Placeholder</div>`).
20. Verify tab renders only when `ceEnabled`, badge updates with `pendingCount`.

Phase 6 — CeIntakeOverlay

21. Create `src/components/CeIntakeOverlay.tsx` — all four states (processing, auto_executed, queued, rejected, error).
22. Wire into `App.tsx` deep-link routing — replace direct navigation with CE routing when configured.
23. Write integration tests for all overlay states.
24. Verify fallback to direct mode works correctly.

Phase 7 — Modify ReceiveScreen and PresentScreen

25. Add CE routing to manual QR scan path in both screens.
26. Add "CE bypassed" informational note for direct mode.
27. Test: scan QR → if CE configured → overlay → auto_executed / queued / fallback.

Phase 8 — Consent Rules Screen and Editor

28. Create `ConsentRulesScreen.tsx` — list, enable/disable toggle, delete.
29. Create `ConsentRuleEditorScreen.tsx` — full multi-step wizard.
30. Wire discovery API call for credential types (for Step 2 and Step 4 dropdowns).
31. Write integration tests for create rule and edit rule flows.

Phase 9 — Queue Screen and Detail

32. Create `ConsentQueueScreen.tsx` — list with filter tabs, quick actions.
33. Create `ConsentQueueDetailScreen.tsx` — full detail, approve/reject, rule creation toggle.
34. Wire `approveQueueItem` with PIN entry for credential offers that require PIN.
35. Write integration tests for approve/reject flows.

Phase 10 — Audit Log

36. Create `AuditLogScreen.tsx` — infinite scroll list with date grouping and action icons.
37. Wire filter tabs to `listAuditEvents()` with `action` filter param.
38. Test empty state, first page, load-more.

Phase 11 — Status Banner and Polish

39. Create `CeStatusBanner.tsx` — show on DashboardScreen conditionally.

40. Wire pending count banner tap to `consent_queue` navigation.
41. Wire disconnected banner retry to `refreshHealth()`.
42. End-to-end pass: configure CE → create rule → scan QR → verify auto_executed → check audit log → approve queued item → verify badge clears.

SECTION 19 — ACCEPTANCE CRITERIA

The implementation is complete when all of the following are true.

Functional

- A user with no CE configured can use the wallet exactly as today — ReceiveScreen and PresentScreen work without any CE-related changes visible.
- A user who configures a CE URL and API key in AccountScreen sees "● Connected" with pending count.
- After configuring CE, the Consent tab appears in the bottom tab bar.
- A deep link arriving at the wallet URL (`?uri=...`) when CE is configured is routed through `POST /consent/intake` and does NOT navigate directly to ReceiveScreen/PresentScreen.
- When intake returns `auto_executed`, the overlay shows a success state and auto-dismisses.
- When intake returns `queued`, the overlay shows a queue-pending state with a "Review Now" button.
- When intake returns `manual_required` or CE is unreachable, the wallet falls back to direct mode transparently.
- A manual QR scan in ReceiveScreen or PresentScreen is also routed through CE when configured.
- The rule editor wizard creates a valid rule for all combinations of: verification/issuance × any/specific credential type × any/specific party × any/explicit fields.
- Editing an existing rule pre-fills all wizard steps with the rule's current values.
- Enabling/disabling a rule from ConsentRulesScreen takes effect immediately (toggle optimistically, revert on error).
- Deleting a rule shows a confirmation sheet before proceeding.
- The queue screen shows all items with `status: "pending"` and correct expiry countdowns.
- Approving a queue item calls the CE API and shows success, then removes item from list.
- Approving a queue item with "create rule" enabled also creates the rule and shows confirmation.
- Rejecting a queue item calls the CE API and removes item from list.

- The Consent tab badge shows the current `pendingCount` and disappears when count is 0.
- The audit log loads paginated events, groups them by date, and load-more works correctly.

UX

- All new screens follow the existing visual language (`#F2F2F7` background, white cards, `#5B4FE9` primary colour, framer-motion transitions).
- All loading states show skeleton placeholders, not just spinners.
- All errors show friendly, human-readable messages.
- All destructive actions show a confirmation bottom sheet.
- The Consent tab only appears when CE is configured and enabled.
- Onboarding Step 3 is shown only once per login session (skipped if `neoke_ce_dismissed` is set).
- The CE status banner on DashboardScreen correctly shows disconnected state and pending count.

Auth and Security

- The raw API key is stored in `localStorage` under `neoke_ce_apikey` at onboarding Step 2 success.
- The API key is cleared from `localStorage` on full logout.
- CE API calls always use `Authorization: ApiKey {key}`, never `Bearer {token}`.
- The bearer token from `AuthContext` is never passed to CE API calls.
- If CE returns 401, the wallet shows a reconnect prompt and disables CE automation.

SECTION 20 — REFERENCE: CE API BASE URL PATTERNS

For the developer: the CE base URL follows no specific format convention beyond being a valid `https://` URL. Examples:

text
```
https://consent.example.com
https://consent-engine.internal.company.com
https://ce.neoke-prod.example.com
https://localhost:3500   ← for local development only
```

All CE API paths are relative to this base:

text
```
{ceBaseUrl}/consent/intake
{ceBaseUrl}/rules
{ceBaseUrl}/rules/{id}
{ceBaseUrl}/queue
{ceBaseUrl}/queue/{id}
{ceBaseUrl}/queue/{id}/approve
{ceBaseUrl}/queue/{id}/reject
{ceBaseUrl}/audit
{ceBaseUrl}/node/{nodeId}/credentials
{ceBaseUrl}/health
```

No path prefix beyond the base URL is required. The CE does not namespace its API by version in the MVP.

SECTION 21 — OPEN QUESTIONS FOR PRODUCT REVIEW

Document these as `TODO` comments in the code:

1. **Scan tab behaviour with CE:** Currently tapping "Scan QR Code" navigates to `ReceiveScreen`. With CE routing added, should the tab still be labelled "Scan QR Code", or should it be relabelled something more neutral like "Scan" since the user no longer directly controls what happens after scanning?
2. **CE API key rotation:** If the user rotates their node API key (new key issued, old one revoked), they must update the CE's stored key in AccountScreen. There is no automatic rotation notification. Should AccountScreen show the last-successful-auth timestamp to help users detect stale keys?
3. **Queue item expiry display:** The CE's default TTL for queued requests is 24 hours. The wallet should show countdown timers ("Expires in 23h 45m"). At what threshold should the "Expiring soon" warning appear? Current proposal: 1 hour. Confirm.
4. **Approval of credential offers that require a PIN:** When `preview.requiresPin: true`, the wallet must ask the user for the PIN before calling `approveQueueItem()`. The CE will pass it to the node. However, if the issuer's PIN has expired, the CE will

return an error. How should the wallet communicate this to the user? Proposal: show "This offer's PIN may have expired — contact the issuer for a new one."

5. **Multiple credentials matching one VP request:** When the CE previews a VP request and `matchedCredentials` contains more than one entry for the same credential type (e.g., two driving licences), the CE picks the most recently issued one. The wallet currently has no way to indicate which credential was used. Should the audit log entry include the credential ID that was selected? Confirm with CE team.
6. **Offline / backgrounded wallet:** The CE processes requests autonomously on the server even when the wallet is closed. When the user next opens the wallet, `refreshHealth()` fires and `pendingCount` is updated. Is the 30-second background poll sufficient for the expected latency requirements, or does the use case require push notifications (future scope)?